AI-Driven AWS Cloud Architecture Recommender

Comprehensive Project Explanation & Presentation Guide

Cloud Architecture Design (VIT) | Guide: Dr. Sunija A P

Person A: Sudipta Majumder (24BCE2882) | **Person B:** P. Giphy Bershan (24BCE2201)

---

1. High-Level Overview: What the System Does

The **AI-Driven AWS Cloud Architecture Design & Recommendation System** is an intelligent, recommendation-only decision-support tool. It takes an open-ended, plain English description of an application (e.g. *'A highly available e-commerce platform for ~100k users'*) and automatically transforms it into an end-to-end, production-grade cloud recommendation.

Stage	Component	Technology	Nature
1. Requirement Parsing	Extracts technical attributes (traffic, availability, budget)	Amazon Bedrock (LLM)	Probabilistic (AI)
2. Architecture Reasoning	Selects reference pattern and catalogue-bounded services	Amazon Bedrock (LLM)	Probabilistic (AI)
3. Graph Validation	Audits allow-lists, connection rules & detects orphans	jsonschema + Python	Deterministic (Code)
4. Diagram Rendering	Renders visual architecture diagram with official AWS icons	Python diagrams.aws + DOT	Deterministic (Code)
5. Cost Estimation	Calculates monthly low/high cost in ap-south-1	Catalogue Pricing Model	Deterministic (Code)
6. Security Review	Audits 15 rules across Well-Architected Framework pillars	Rule-based Audit Engine	Deterministic (Code)

Key Engineering Principle: *'LLMs for language understanding; deterministic code for math, security, and graphics.'* Crucially, the system is **recommendation-only**: it never executes AWS CLI commands or deploys live infrastructure, ensuring 100% safety against accidental infrastructure charges or misconfigurations.

---

2. Person A Guide: Sudipta Majumder (AI Core & Pipeline)

Person A owns the **intelligence layer, schemas, prompts, guardrails, and evaluation**. Files owned: parser.py, reasoner.py, llm.py, validator.py, schemas/, catalogue/, prompts/, eval/.

A. Exactly What Sudipta Built:

- **Bedrock LLM Wrapper (llm.py):** Dynamic configuration reading BEDROCK_MODEL_ID and AWS_REGION from .env. Actionable exception handling for AccessDenied, throttling, and timeouts. Model-keyed on-disk cache (.cache/) and real-time token tracking to display exact prompt token metrics on slides.
- **Prompt Engineering & Few-Shot Injection (parser.py, reasoner.py):** Loaded 4 comprehensive few-shot examples (3-tier web, serverless REST, static CDN, batch pipeline). Prompt injection defense wrapping user inputs in <user_requirement> tags. Automated 2-step schema repair-and-retry.

- **Offline Demo Safety Net (examples/, run.py):** 3 complete cached offline test cases (ecommerce, serverless_api, static_site) allowing full demo execution without AWS/Internet.
- **Evaluation Benchmark Harness (eval/run_eval.py):** Quantitative metric calculation: Service Selection Accuracy (100%), Precision (100%), Pattern Match % (100%), and Architecture Validity (100%) against ground-truth blueprints.
- **Scenario Syntax Checker (eval/check_scenarios.py):** Shared checker validating ground-truth scenarios.

B. Sudipta's Presentation Speech (What to say during presentation):

"Good morning. I am Sudipta Majumder, and I developed the AI Core and validation pipeline. When designing with LLMs, the primary hazard is hallucination—models invent fake AWS services or wire invalid networks. To solve this, I designed a two-stage pipeline strictly constrained by JSON Schema contracts. First, my Requirement Parser extracts structured attributes while treating user text inside <user_requirement> tags as untrusted data to neutralize prompt injection attacks. Second, my Reasoning Engine maps requirements strictly to our curated catalogue.yaml. If an LLM attempts to connect an ALB directly to S3 or invent a database, our validator intercepts and rejects it immediately. Finally, I built an automated evaluation harness in eval/run_eval.py that measures 100% service recall and 100% precision against official AWS reference blueprints."

C. Questions Examiners May Ask Sudipta:

Q: Why two separate LLM calls instead of one single prompt?
Answer: Single-prompt generation suffers from task overload. Splitting into parsing and reasoning enforces modularity, schema-bound boundaries, and higher accuracy.

Q: How do you prevent prompt injection?
Answer: User text is quarantined in XML delimiters and explicitly instructed as passive untrusted data, verified by automated security injection tests.

---

3. Person B Guide: P. Giphy Bershan (Downstream, Infra & UI)

Person B owns the **deterministic analysis, pricing, diagramming, deployment architecture, and UI**. Files owned: diagram.py, cost.py, security.py, app/, scripts/, deployment guides.

A. Exactly What Giphy Built:

- **Deterministic Diagram Renderer (diagram.py):** Generates production architecture diagrams using diagrams.aws.* iconography, clustered logically by category (Compute, Network, Storage, Database), with automated Graphviz DOT fallback.
- **Well-Architected Security Auditor (security.py):** Evaluates 15 deterministic checks across Security, Reliability, Cost Optimization, and Operational Excellence (flagging public S3, missing TLS, missing Multi-AZ, etc.).
- **Regional Cost Estimator (cost.py):** Computes monthly low/high cost ranges based on ap-south-1 prices and explicit usage assumptions.
- **System Deployment Architecture:** Designed the serverless hosting stack for our own application: S3 + CloudFront (SPA), API Gateway HTTP API, Lambda, Bedrock API, and DynamoDB ($0 idle operational cost in Free Tier).
- **Streamlit Interactive Interface (app/streamlit_app.py):** Web application featuring Services, Diagram PNG, Cost breakdown, Security checks tabs, and an offline Demo Mode toggle.

B. Giphy's Presentation Speech (What to say during presentation):

"Good morning. I am P. Giphy Bershan, and I developed the downstream analysis, cloud deployment architecture, and user interface. Once Sudipta's AI core produces a validated Architecture JSON, my downstream modules execute

deterministically with zero additional LLM costs. My diagram module parses the JSON graph into clusters and maps official AWS icons via the Python diagrams library. My security module audits the design against 15 Well-Architected rules, flagging issues like unencrypted storage or missing multi-AZ redundancy. Finally, our system's own deployment uses a completely serverless stack—S3, CloudFront, API Gateway, and Lambda—which costs literally zero dollars at idle within the AWS Free Tier, with Bedrock token charges totaling under $10 a month."

C. Questions Examiners May Ask Giphy:

Q: Why not let the LLM draw the diagram in Mermaid or SVG?

Answer: Published benchmarks (DiagramAI 2025) prove LLMs suffer high syntax and alignment error rates when drawing directly. Using deterministic code on structured JSON guarantees 100% syntactically valid diagrams.

Q: Why is the system strictly recommendation-only?

Answer: Autonomous deployment introduces severe financial and operational hazards. An LLM mistake could provision expensive GPU clusters or create open security holes. Cloud governance mandates human review.